

MÓDULO 9 — Sistema de Comunicación (parte 3, continuación)

Se va a ajustar la opción de "My Tickets", actualmente, si un administrador ingresa a "My Tickets", ve el listado completo en el sistema, que estaría duplicado con lo que se reporta en el Dashboard.

Si un Administrador o un Técnico entra a "My Tickets", lo que realmente esperan ver es su **"Bandeja de Entrada Personal"**, que incluye tanto los Tickets que han reportado como los que tienen pendiente por resolver. Se procede a ajustar la lógica para que "My Tickets" se convierta en el centro de trabajo personal.

1. Actualización en el Servicio (TicketService.cs)

Se agrega el método GetMyWorkAndRequestsAsync, en el que se modifica la consulta LINQ para que use un operador **OR (||)**. De esta forma, se traen los tickets donde el usuario sea el autor/creador **O** sea el técnico asignado para resolverlo.

Editar el Archivo: /Services/TicketService.cs

```
24
25 // trae los tickets donde el usuario es el autor O el técnico asignado.
26 1 reference public async Task<List<Ticket>> GetMyWorkAndRequestsAsync(string userId)
27 {
28     using var context = dbFactory.CreateDbContext();
29     return await context.Tickets
30         .Include(t => t.Category) // Para mostrar el nombre de la categoría en la lista
31         .Include(t => t.Author)    // Para mostrar el email del autor
32         .Include(t => t.Technician) // Para mostrar el email del técnico asignado
33         // LÓGICA HÍBRIDA: Lo que reporté O lo que debo resolver
34         .Where(t => t.AuthorId == userId || t.TechnicianId == userId)
35         .OrderByDescending(t => t.CreatedAt)
36         .ToListAsync();
37 }
38
39 // --- MÓDULO 4: MÉTODOS DE CREACIÓN ---
```

2. Actualización en la Interfaz (MyTickets.razor)

En la tabla, ahora es importante diferenciar visualmente si el usuario es el "creador/dueño" o el que "resuelve/atiende" el ticket. Se agrega una pequeña etiqueta para dar ese contexto y diferenciarlos.

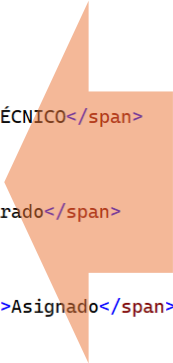
Archivo: /Components/Pages/Tickets/MyTickets.razor (Fragmento del @code)

```
83 @code {
84     private List<Ticket>? userTickets = new(); // Inicializamos como lista vacía para evitar null reference
85     private string? userId; // Variable para almacenar el User ID del usuario autenticado
86
87     // En este método se obtiene el User ID del usuario autenticado y luego se cargan
88     // solo los tickets relacionados con ese usuario
89     protected override async Task OnInitializedAsync()
90     {
91         var authState = await AuthProvider.GetAuthenticationStateAsync();
92         var user = authState.User;
93         userId = user.FindFirst(ClaimTypes.NameIdentifier)?.Value; // Obtenemos el User ID del usuario autenticado
94
95         if (!string.IsNullOrEmpty(userId))
96         {
97             // Se usa el nuevo método que devuelve sólo los tickets relacionados con el usuario
98             userTickets = await TicketService.GetMyWorkAndRequestsAsync(userId);
99         }
100     }
101
102     // Método para obtener la clase CSS según el estado del ticket
103     private string GetStatusClass(TicketStatus status) => status switch
104     {
105         TicketStatus.Open => "bg-info text-dark",
106         TicketStatus.InProgress => "bg-warning text-dark",
107         TicketStatus.Resolved => "bg-success",
108         _ => "bg-secondary"
109     };
110 }
111
```

Editar Archivo: /Pages/Tickets/MyTickets.razor (Fragmento de la Tabla HTML):

```
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77

<table class="table table-hover align-middle mb-0">
  <thead class="bg-light">
    <tr>
      <th class="ps-4">Título</th>
      <th>Estado</th>
      <th>Prioridad</th>
      <th>Acciones</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var ticket in userTickets)
    {
      <tr>
        <td>
          <div class="fw-bold">@ticket.Title</div>
          @* ETIQUETA DE ROL EN EL TICKET *@
          @if (ticket.AuthorId == userId && ticket.TechnicianId == userId)
          {
            <span class="badge bg-dark small" style="font-size: 0.6rem;">AUTOR Y TÉCNICO</span>
          }
          else if (ticket.AuthorId == userId)
          {
            <span class="badge bg-secondary small" style="font-size: 0.6rem;">Generado</span>
          }
          else
          {
            <span class="badge bg-info text-dark small" style="font-size: 0.6rem;">Asignado</span>
          }
        </td>
        <td><span class="badge bg-secondary">@ticket.Status</span></td>
        <td>@ticket.Priority</td>
        <td>
          <button class="btn btn-sm btn-success"
            @onclick="@(() => Navigation.NavigateTo($"/tickets/manage/{ticket.Id}"))">
            Work on it
          </button>
        </td>
      </tr>
    }
  </tbody>
</table>
```



Actualizar Repositorio (Requerido)

Implementación de "Mensajes del Sistema" Automáticos

El "hilo" de mensajes del ticket debe actuar como una **bitácora de auditoría (Audit Log)**. En este sentido, si un técnico cambia el estado a "In Progress", el cliente debería ver un mensaje automático que reporte el cambio: *"[SISTEMA]: El técnico ha iniciado el trabajo"*. Actualmente, el código solo registra los mensajes manuales, reportados por las personas. Vamos a automatizar para que cada acción importante se reporte en la conversación.

Para lograr esto, se modifica/edita el `TicketService.cs`. Cada vez que llamemos a un método de actualización, el servicio se encargará de insertar un comentario automático.

1. Actualización en `TicketService.cs`

Modificaremos los métodos de asignación y cambio de estado para que disparen el registro en la tabla de comentarios.

Editar el Método `UpdateTicketStatusAsync`

```
146 // Método para actualizar el estado de un ticket, con lógica para registrar eventos importantes
147 public async Task UpdateTicketStatusAsync(int ticketId, TicketStatus newStatus, string? newTechId = null, string? performerId = null)
148 {
149     using var context = dbFactory.CreateDbContext();
150
151     var ticket = await context.Tickets
152         .Include(t => t.Technician)
153         .FirstOrDefaultAsync(t => t.Id == ticketId);
154
155     if (ticket != null)
156     {
157         string systemLog = "";
158         var oldStatus = ticket.Status;
159         var oldTechId = ticket.TechnicianId;
160
161         // ESCENARIO 1: Solo cambio de Técnico (Asignación por Admin)
162         if (!string.IsNullOrEmpty(newTechId) && newTechId != oldTechId)
163         {
164             ticket.TechnicianId = newTechId;
165             // No cambiamos el status aquí, se mantiene el que tiene (Open)
166             systemLog = $"[SISTEMA] El Administrador ha asignado este ticket al equipo técnico.";
167         }
168
169         // ESCENARIO 2: Cambio de Estado (Técnico inicia o resuelve)
170         if (newStatus != oldStatus)
171         {
172             ticket.Status = newStatus;
173             systemLog = string.IsNullOrEmpty(systemLog)
174                 ? $"[EVENTO] El estado cambió de {oldStatus} a {newStatus}."
175                 : systemLog + $" El estado actual es {newStatus}.";
176         }
177
178         if (!string.IsNullOrEmpty(systemLog))
179         {
180             var systemComment = new TicketComment
181             {
182                 TicketId = ticketId,
183                 UserId = performerId ?? (newTechId ?? ticket.AuthorId),
184                 Content = systemLog,
185                 CreatedAt = DateTime.UtcNow
186             };
187
188             context.TicketComments.Add(systemComment);
189             ticket.UpdatedAt = DateTime.UtcNow;
190             await context.SaveChangesAsync();
191         }
192     }
193 }
```

Editar el Método `AssignTicketAsync`

```
194 // Método específico para que el admin asigne un técnico a un ticket, con registro de evento
195 public async Task AssignTicketAsync(int ticketId, string technicianId, string adminId)
196 {
197     using var context = dbFactory.CreateDbContext();
198     var ticket = await context.Tickets
199         .Include(t => t.Technician)
200         .FirstOrDefaultAsync(t => t.Id == ticketId);
201
202     if (ticket != null)
203     {
204         ticket.TechnicianId = technicianId;
205         ticket.Status = TicketStatus.InProgress;
206
207         var systemMessage = new TicketComment
208         {
209             TicketId = ticketId,
210             UserId = adminId,
211             Content = $"[SISTEMA] El Administrador ha asignado este ticket al equipo técnico.",
212             CreatedAt = DateTime.UtcNow
213         };
214
215         context.TicketComments.Add(systemMessage);
216
217         await context.SaveChangesAsync();
218     }
219 }
220
221 // --- MÓDULO 7: MÉTODOS DE REPORTES Y ESTADÍSTICAS ---
```

2. Ajuste Visual en ManageTicket.razor

Para que los mensajes del sistema no se confundan con una respuesta humana, se aplica estilo "neutral" (centrados y en cursiva).

Editar dentro del bucle @foreach de los comentarios:

```
63      @foreach (var c in comments)
64      {
65          bool isSystem = c.Content.StartsWith("["); // Detectar prefijos de sistema
66          bool isClient = c.UserId == ticket.AuthorId;
67          bool isMe = c.UserId == currentUserId;
68
69          if (isSystem)
70          {
71              @* MENSAJE DE SISTEMA / AUDITORÍA *@
72              <div class="text-center my-3">
73                  <span class="badge rounded-pill bg-light text-muted border px-3">
74                      <i class="bi bi-info-circle me-1"></i> @c.Content.Replace("[EVENTO]", "").Replace("[SISTEMA]", "")
75                  </span>
76                  <div style="font-size: 0.6rem;" class="text-muted mt-1">@c.CreatedAt.ToLocalTime().ToString("t")</div>
77              </div>
78          }
79          else
80          {
81              <div class="d-flex mb-4 @(isMe ? "justify-content-end" : "justify-content-start")">
82                  <div class="shadow-sm rounded-3 @(isClient ?
83                      "bg-white border-start border-4 border-secondary" :
84                      "bg-primary text-white border-start border-4 border-info)"
85                      style="max-width: 80%; min-width: 200px; padding: 12px;">
86
87                      <div class="d-flex justify-content-between align-items-center mb-2">
88                          <span class="badge @(isClient ? "bg-secondary" : "bg-info text-dark")
89                              text-uppercase" style="font-size: 0.65rem;">
90                              @(isClient ? "Cliente" : "Soporte Técnico")
91                          </span>
92                          <small class="opacity-75 ms-2" style="font-size: 0.7rem;">
93                              @c.CreatedAt.ToLocalTime().ToString("t")
94                          </small>
95                      </div>
96
97                      <div class="fw-bold small mb-1 @(isClient ? "text-primary" : "text-white-50")">
98                          @c.User?.Email
99                      </div>
100                     <div class="text-break">@c.Content</div>
101                 </div>
102             </div>
103         }
104     }
```

Editar la Sección de Botones Dinámicos (HTML): El botón de "Iniciar Trabajo" debe aparecer si el ticket está en Open, sin importar si ya tiene técnico o no.

```
142     <div class="btn-group w-100 shadow-sm">
143         @if (ticket.Status == TicketStatus.Open)
144         {
145             @* Solo el técnico asignado (o el admin) puede iniciar el trabajo *@
146             <button class="btn btn-warning py-2" @onclick="() => UpdateStatus(TicketStatus.InProgress)">
147                 <i class="bi bi-play-fill"></i> Iniciar Trabajo
148             </button>
149         }
150
151         @if (ticket.Status == TicketStatus.InProgress)
152         {
153             <button class="btn btn-success py-2" @onclick="() => UpdateStatus(TicketStatus.Resolved)">
154                 <i class="bi bi-check-all"></i> Marcar como Resuelto
155             </button>
156         }
157         @if (ticket.Status == TicketStatus.Resolved)
158         {
159             <div class="alert alert-info w-100 mb-0 py-2 text-center">
160                 <i class="bi bi-lock-fill"></i> El ticket está cerrado y resuelto.
161             </div>
162         }
163     </div>
```

Editar Archivo: /Pages/Tickets/ManageTickets.razor (Fragmento del @code)

```
225 private async Task AssignTicket()
226 {
227     if (string.IsNullOrEmpty(selectedTechId)) return;
228
229     // Mantenemos el status actual (ticket.Status) en lugar de forzar InProgress
230     await TicketService.UpdateTicketStatusAsync(TicketId, ticket!.Status, selectedTechId, currentUserId);
231
232     feedbackMessage = "Técnico asignado. El ticket está a la espera de ser iniciado.";
233     await LoadData();
234 }
235
236 private async Task UpdateStatus(TicketStatus status)
237 {
238     // Pasamos currentUserId como el técnico que está operando
239     await TicketService.UpdateTicketStatusAsync(TicketId, status, null, currentUserId);
240
241     feedbackMessage = $"Estado actualizado a {status}.";
242     await LoadData();
243 }
244 }
```

Resultado:

Con los cambios realizados hemos logrado:

1. **Trazabilidad:** Si un ticket se cierra sin solución, el Admin puede ver exactamente en qué minuto cambió de "In Progress" a "Resolved" y quién lo hizo.
2. **Auditabilidad Real:** Al guardar el performerId, el historial no solo dice qué pasó, sino que queda vinculado internamente a quién hizo clic en el botón.
3. **Transparencia con el Cliente:** El empleado recibe retroalimentación inmediata de que "algo está pasando" con su solicitud, incluso si el técnico no ha escrito un mensaje manual todavía.
4. **Patrón de Diseño:** Se aplica el concepto de **Domain Events** (Eventos de Dominio) de una forma simplificada y práctica.

Ciclo de Vida del Ticket

1. **Creación:** El ticket nace como Open y sin técnico.
2. **Asignación:** El Admin asigna un técnico. El ticket sigue Open. El log dice: " El Administrador ha asignado este ticket al equipo técnico."
3. **Recepción:** El técnico ve el ticket en su lista. Ve que está en Open.
4. **Ejecución:** El técnico pulsa "Iniciar Trabajo". El log dice: "*El estado cambió de Open a InProgress*".
5. **Cierre:** Solo entonces aparece el botón de "**Marcar como Resuelto**". El log dice: " El estado cambió de InProgress a Resolved."

Actualizar Repositorio (Requerido)

```
Developer PowerShell
+ Developer PowerShell v |  |  | 
                                git status

On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   SupportTicketsSystem2026/Components/Pages/Tickets/ManageTicket.razor
        modified:   SupportTicketsSystem2026/Components/Pages/Tickets/MyTickets.razor
        modified:   SupportTicketsSystem2026/Services/TicketService.cs

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status

On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   SupportTicketsSystem2026/Components/Pages/Tickets/ManageTicket.razor
        modified:   SupportTicketsSystem2026/Components/Pages/Tickets/MyTickets.razor
        modified:   SupportTicketsSystem2026/Services/TicketService.cs

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Modulo 9(part3): Implementar la logica para la Bandeja de Entrada Personal
[main c40885b] Modulo 9(part3): Implementar la logica para la Bandeja de Entrada Personal y los Mensajes del Sistema Automáticos
 3 files changed, 150 insertions(+), 36 deletions(-)
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 24 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 3.07 KiB | 3.07 MiB/s, done.
Total 10 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), completed with 7 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
   111951c..c40885b  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```